FDU 数据库 1. 关系数据库

本文参考以下教材：

- Database System Concepts (A. Silberschatz, H. Korth, S. Sudarshan) Chapter $1, 2$
- 数据库系统概念 (A. Silberschatz, H. Korth, S. Sudarshan, 杨冬青等译) 第 $1, 2$ 章

欢迎批评指正!

1.1 An Introduction

数据库管理系统 (DataBase-Management System, DBMS)
由一个互相关联的数据集合 (称为**数据库**, database) 和用来访问这些数据的程序组成.
其设计目的是提供一种高效存取数据库信息的途径:
它隐藏了关于数据存储和维护的某些细节，为用户提供数据的抽象视图.

使用传统的文件系统管理大量信息会有以下弊端:

- **① 数据冗余和不一致性 (data redundancy and inconsistency)**
  同一数据可能具有多个副本，这将导致存储和访问开销增大.
  如果对该数据的更新操作 没有应用到所有副本上，那么还会造成数据的不一致性.
- **② 数据访问困难 (difficulty in accessing data)**
  传统的文件系统只支持使用地址来访问数据，不能满足用户按需检索数据的需求.
- **③ 并发访问异常 (concurrent-access anomaly)**
  并发的更新操作可能会相互影响，但传统的文件系统无法提供这种监管.
- **④ 安全性问题 (security problem)**
  我们需要为不同的用户设置不同的访问权限，在这一点上，传统的文件系统不够灵活.

上述问题促使了 20 世纪中叶数据库系统的发展和基于文件的应用向基于数据库的应用的变迁.

数据库的结构基础是**数据模型** (data model):
一个描述数据、数据联系、数据语义以及一致性约束的概念工具的集合.
本课程主要介绍**关系模型** (relational model)，因为它是大多数数据库系统的基础.

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Figure 1.1 A sample relational database.

数据库系统提供**数据定义语言** (Data-Definition Language, DDL) 来定义数据库模式，
并提供**数据操纵语言** (Data-Manipulation Language, DML) 来表达数据库的查询和更新.
它们是数据库语言 (例如 SQL 语言) 的不同部分.
几乎所有的关系数据库系统都使用 SQL 语言.

1.2 关系数据库

关系模型是商用数据处理应用的主要数据模型，它能大大简化程序员的工作.

1.2.1 结构

关系数据库 (relational database) 是**表** (table) 的集合，每张表都有一个唯一的名称.

- 例如 instructor 表的每一行都记录了一位教师的信息，包括 ID、name、dept_name 和 salary.
  (每位教师通过 ID 列的取值来进行标识)
- 例如 course 表的每一行都记录了一门课程的信息，包括 course_id、title、dept_name 和 credits.
  (每门课程通过 course_id 列的取值来进行标识)
- 例如 prereq 表的每一行由一个课程标识对组成: 第二门课程是第一门课程的先修课.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure 2.2 The *course* relation.

course_id	*prereq_id*
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure 2.3 The *prereq* relation.

一般来说，表中的一行代表了一组值之间的某种联系.
在关系模型中，**关系** (relation) 被用来指代表，**元组** (tuple) 被用来指代行，**属性** (attribute) 指代的是表中的列.
元组的个数称为**基数** (cardinality)，属性的个数称为**元数** (arity)

- 我们称一个关系的特定实例为**关系实例** (relation instance)
- 由于关系是元组的集合，所以元组在关系中出现的顺序是无关紧要的.
  我们一般按关系的第一个属性的次序来排列元组.
- 关系的每个属性都存在一个取值集合，称为**域** (domain)
  我们要求所有属性的域都是**原子的** (atomic)，即其中的元素可视为不可再分的单元.
  空值 (null value) 表示值未知或不存在，我们应尽量避免使用空值.
  公共属性是将不同关系的元组联系起来的一种方式.

1.2.2 模式

关系的概念对应于程序设计中变量的概念，
而**关系模式** (relation schema) 的概念对应于程序设计中的类型定义的概念.

一般说来，一个关系模式由一个属性列表及各属性所对应的域组成.

- 例如 department 关系 (用于描述各院系所在建筑和预算) 的模式为:

$$\text{department}(\text{dept\_name},\ \text{building},\ \text{budget})$$

- 例如 section 关系 (用于描述每个课程段的情况) 的模式为:

$$\text{section}(\text{course\_id},\ \text{sec\_id},\ \text{semester},\ \text{year},\ \text{building},\ \text{room\_number},\ \text{time\_slot\_id})$$

- 例如 teacher 关系 (用于描述教师和所教课程之间的联系) 的模式为:

$$\text{teaches}(\text{ID},\ \text{course\_id},\ \text{sec\_id},\ \text{semester},\ \text{year})$$

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure 2.5 The *department* relation.

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure 2.6 The *section* relation.

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure 2.7 The *teaches* relation.

总之，一个大学数据库要维护以下关系:

$$\begin{aligned}
&\text{instructor}(\text{ID}, \text{name}, \text{dept_name}, \text{salary})\\
&\text{department}(\text{dept_name}, \text{building}, \text{budget})\\
&\text{course}(\text{course_id}, \text{title}, \text{dept_name}, \text{credits})\\
&\text{section}(\text{course_id}, \text{sec_id}, \text{semester}, \text{year}, \text{building}, \text{room_number}, \text{time_slot_id})\\
&\text{prereq}(\text{course_id}, \text{prereq_id})\\
&\text{teaches}(\text{ID}, \text{course_id}, \text{sec_id}, \text{semester}, \text{year})\\
&\text{student}(\text{ID}, \text{name}, \text{dept_name}, \text{tot_cred})\\
&\text{advisor}(\text{s_id}, \text{i_id})\\
&\text{takes}(\text{ID}, \text{course_id}, \text{sec_id}, \text{semester}, \text{year}, \text{grade})\\
&\text{classroom}(\text{building}, \text{room_number}, \text{capacity})\\
&\text{time_slot}(\text{time_slot_id}, \text{day}, \text{start_time}, \text{end_time})
\end{aligned}$$

1.2.3 键

一个元组的所有属性值必须能够唯一标识元组.
换言之，一个关系中不能有两个元组在所有属性上取值完全相同.

键 (key) 由若干个属性组成，分为以下几种类型:
(注意: 键是整个关系的性质，而不是单个元组的性质)

- **① 超键 (Super Key)**
  超键可以在关系中唯一地标识一个元组.
  它可以包含不必要的属性，这些属性对于唯一标识是多余的.
  换言之，超键的任意超集也是超键.
  例如在 instructor 表中，$\{\text{ID}, \text{name}\}$ 的组合可以作为一个超键.
  但仅 ID 本身也足以唯一地标识每个教师，使得 name 对于唯一性来说是不必要的.
- **② 候选键 (Candidate Key)**
  候选键是没有多余属性的超键，即其每个属性对于唯一性来说都是必要的.
  换言之，候选键的任意真子集都不能构成超键.
- **③ 主键 (Primary Key)**
  在若干个候选键中，我们选择一个主键来唯一地标识关系中的元组.
  主键的值应当极少变化，并且不能包含空值，这称为**主键约束**.
  按照惯例，我们应将关系模式的主键属性列于其他属性之前，并加上下划线.
- **④ 外键 (Foreign Key)**
  简单来说，外键是另一个关系中的主键，用于建立不同关系之间的联系.
  从关系 r_1 的属性集 A 到关系 r_2 的主键 B 的**外键约束**意味着:
  在任何数据库实例中，r_1 中每个元组在 A 上的取值也必须是 r_2 中某个元组在 B 上的取值.
  我们称属性集 A 为从 r_1 引用 r_2 的**外键**，称 r_1 为**引用关系**，称 r_2 为**被引用关系**.
  例如 instructor 表的 dept_name 属性就是从 instructor 表引用 department 表的外键，
  其中 dept_name 属性是 department 表的主键.

  外键约束的泛化是**引用完整性约束**，它放松了被引用属性构成被引用关系主键的要求，
  只需保证引用是有意义的即可 (即不能引用不存在的属性值)
  例如我们要求 section 表的 time_slot_id 属性中的值必须在 time_slot 表的 time_slot_id 属性中出现过.
  当今的数据库系统通常只支持外键约束，而不支持非外键约束的引用完整性约束.

现在我们对大学数据库中每个关系的主键进行标注:

instructor(ID, name, dept_name, salary)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
prereq(course_id, prereq_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
advisor(s_id, i_id)
takes(ID, course_id, sec_id, semester, year, grade)
classroom(building, room_number, capacity)
time_slot(time_slot_id, day, start_time, end_time)

最后我们再补充对外键的标注:

instructor(ID, name, dept_name, salary)
department(dept_name, building, budget)
course(course_id, title, dept_name(FK to department), credits)
section(course_id(FK to course), sec_id, semester, year, {building, room_number}(FK to classroom), time_slot_id)
prereq(course_id(FK to course), prereq_id(FK to course))
teaches(ID(FK to instructor), {course_id, sec_id, semester, year}(FK to section))
student(ID, name, dept_name, tot_cred)
advisor(s_id(FK to student), i_id)
takes(ID(FK to student), {course_id, sec_id, semester, year}(FK to section), grade)
classroom(building, room_number, capacity)
time_slot(time_slot_id, day, start_time, end_time)

1.2.4 模式图

数据库的逻辑设计称为**数据库模式** (database schema)，其实例称为**数据库实例** (database instance)
一个带有主键和外键约束的数据库模式可以用**模式图** (schema diagram) 表示.
以大学数据库为例:

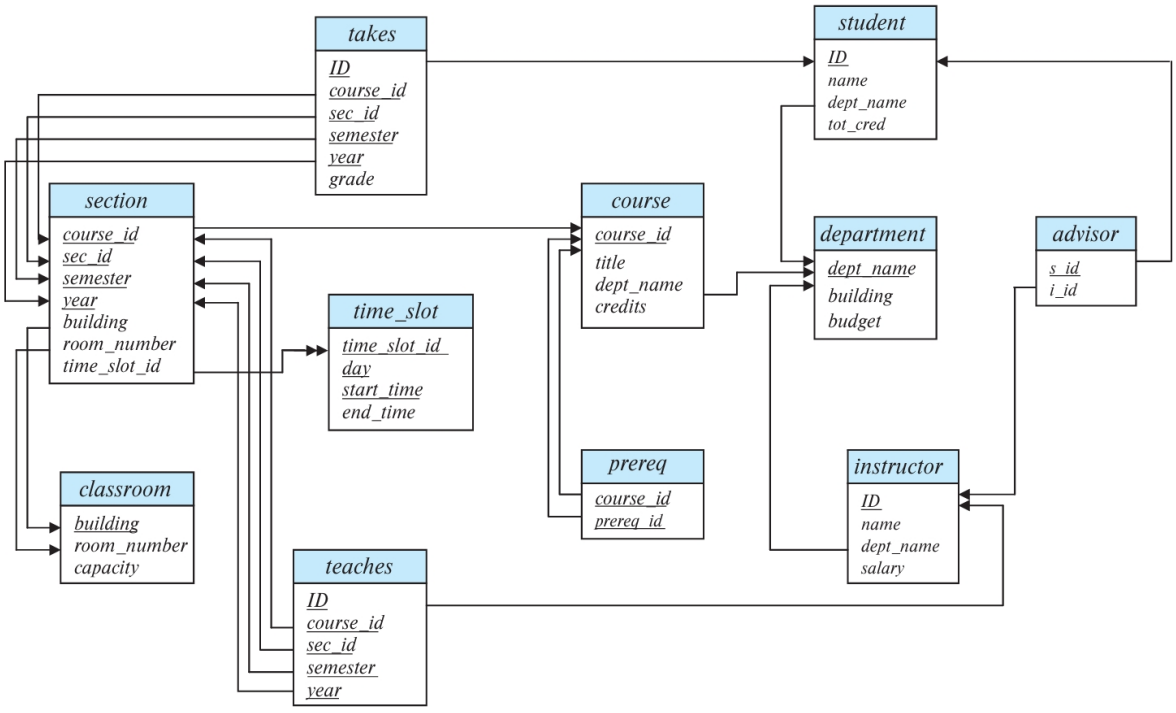
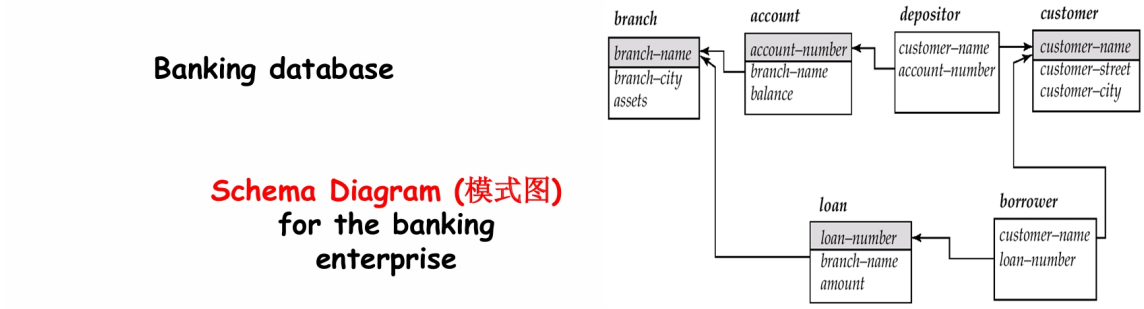


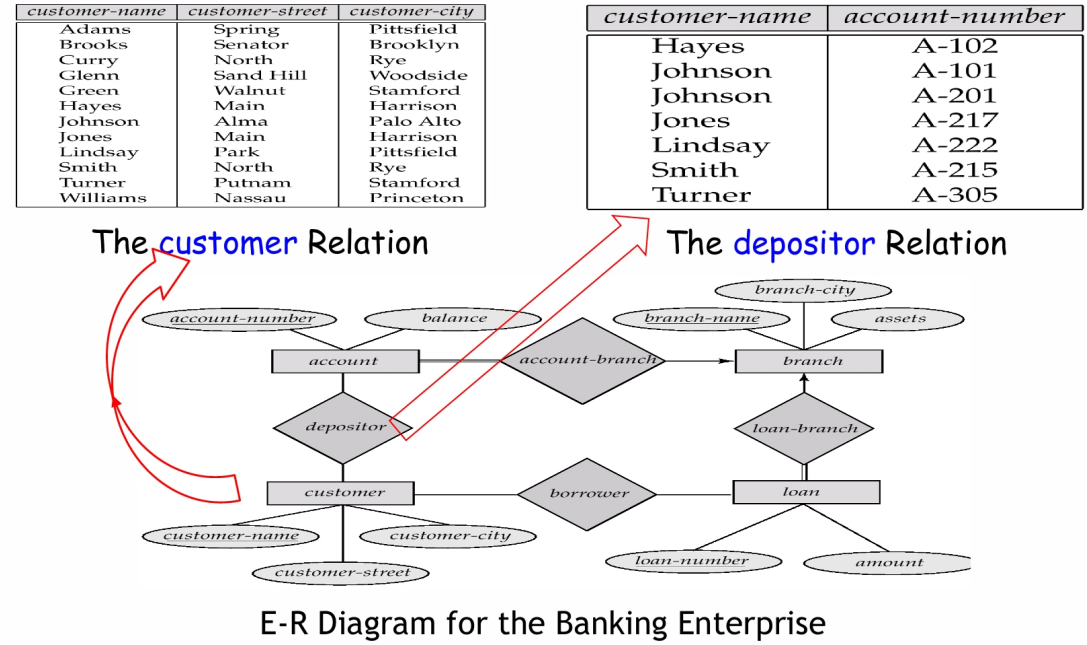
Figure 2.9 Schema diagram for the university database.

以银行数据库为例:

branch (branch_name, branch_city, assets)
customer (customer_name, customer_street, customer_city)
account (account_number, branch_name, balance)
loan (loan_number, branch_name, amount)
depositor (customer_name, account_number)
borrower (customer_name, loan_number)



数据库模式还可用**实体-关系图** (Entity-Relation Diagram) 来表示 (后面我们会单独介绍)
以银行数据库为例:



customer-name	customer-street	customer-city
Adams	Spring	Pittsfield
Brooks	Senator	Brooklyn
Curry	North	Rye
Glenn	Sand Hill	Woodside
Green	Walnut	Stamford
Hayes	Main	Harrison
Johnson	Alma	Palo Alto
Jones	Main	Harrison
Lindsay	Park	Pittsfield
Smith	North	Rye
Turner	Putnam	Stamford
Williams	Nassau	Princeton

The customer Relation

customer-name	account-number
Hayes	A-102
Johnson	A-101
Johnson	A-201
Jones	A-217
Lindsay	A-222
Smith	A-215
Turner	A-305

The depositor Relation

E-R Diagram for the Banking Enterprise

1.3 关系代数

关系代数 (Relational Algebra, RA) 是 SQL 查询语言的基础.
它由一组运算构成，这些运算接受一个或两个关系作为输入，并生成一个新的关系作为结果.

- 一元运算: 选择、投影和更名运算等
- 二元运算: 并、直积和集差运算等

值得注意的是，由于关系是元组的集合，因此理论上关系不能包含重复的元组.
在讨论正式的关系代数时，如集合的数学定义所要求的那样，我们要去除重复项.
但在实践中，我们将关系代数扩展到多重集合上，允许关系包含重复的元组.

A procedural language consisting of a set of operations that take one or more relations as input and produce a new relation as the result

Six basic operations

- Select (选择)；水平选择，选择行/元组
- Project (投影)；垂直选择，选择列/属性
- Union (集合并)
- Set difference (集合差)
- Cartesian product (笛卡尔积)
- Rename (重命名)

These operators take one or two relations as inputs and give a new relation as a result

Additional operations

- Set intersection (集合交)
- Natural join (自然连接)
- Outer join (外连接)
- Division (除)
- Assignment (赋值)

Additional operations do not add any power to the relational algebra, but simplify common queries

- Generalized Projection (广义投影)
- Aggregate Functions (聚合函数)

1.3.1 选择

选择运算选出满足给定谓词的元组，用 $\sigma(\cdot)$ 表示，**谓词** (predicate) 作为下标列出.

$$\sigma_\theta(r) := \{t : t \in r \text{ and } \theta(t) \text{ is true}\}$$

其中谓词 $\theta(\cdot)$ 的表达式由 $\wedge, \vee, \neg, =, \neq, <, >, \leq, \geq$ 构成.
例如从 instructor 表中选出物理系教师对应的元组:

$$\sigma_{\text{dept\_name}=\text{"Physics"}}(\text{instructor})$$

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Figure 2.10 Result of $\sigma_{dept\_name="Physics"}$ (*instructor*).

1.3.2 投影

投影运算可用于过滤关系中的特定属性，用 $\Pi(\cdot)$ 表示，需要保留的属性作为下标列出.
例如从 instructor 表中选出 ID、name 和 salary 属性:

$$\Pi_{\text{ID,name,salary}}(\text{instructor})$$

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Figure 2.11 Result of $\Pi_{ID,\ name,\ salary}(instructor)$.

对投影的列进行一些算术运算，就得到**广义投影**:

Generalized Projection

Extends the projection operation by allowing arithmetic functions to be used in the projection list $\Pi_{F_1,F_2,\ldots,F_n}(E)$

- E is any relational-algebra expression
- Each of $F_1, F_2, \ldots, F_n$ is a arithmetic expression involving constants and attributes in the schema of E

Given relation *credit_info(customer_name, limit, credit_balance)*, find how much more each person can spend:

- $\Pi_{customer\_name,\ limit-credit\_balance}(credit\_info)$

1.3.3 集合运算

参与集合运算的关系 $r(R)$ 和 $s(S)$ 的属性集必须是**相容的** (compatible)，
即属性的个数相同，且属性的域对应相同.
并运算用 $\cup$ 表示，交运算用 $\cap$ 表示，集差运算用 $-$ 表示.

- ① 查找至少在 2017 年秋季学期或 2018 年春季学期其中之一开设的所有课程的集合:

$$\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Fall"}\wedge\text{year}=2017}(\text{section})\right)\bigcup\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Spring"}\wedge\text{year}=2018}(\text{section})\right)$$

- ② 查找在 2017 年秋季学期和 2018 年春季学期都开设的所有课程的集合:

$$\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Fall"}\wedge\text{year}=2017}(\text{section})\right)\bigcap\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Spring"}\wedge\text{year}=2018}(\text{section})\right)$$

- ③ 查找在 2017 年秋季学期开设，但在 2018 年春季学期未开设的所有课程的集合:

$$\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Fall"}\wedge\text{year}=2017}(\text{section})\right)-\Pi_{\text{course\_id}}\left(\sigma_{\text{semester}=\text{"Spring"}\wedge\text{year}=2018}(\text{section})\right)$$

交运算可由集差运算表示:

$$\begin{aligned} r \cap s &= r - (r - s) \\ &= s - (s - r) \end{aligned}$$

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure 2.6 The *section* relation.

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Figure 2.14 Courses offered in either Fall 2017, Spring 2018, or both semesters.

course_id
CS-101

Figure 2.15 Courses offered in both the Fall 2017 and Spring 2018 semesters.

course_id
CS-347
PHY-101

Figure 2.16 Courses offered in the Fall 2017 semester but not in Spring 2018 semester.

1.3.4 直积

直积又称 Descartes 积，用记号 $\times$ 表示，它可用于结合两个关系的信息.
考虑 instructor 表和 teaches 表的直积:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure 2.7 The *teaches* relation.

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Figure 2.12 Result of the Cartesian product *instructor* $\times$ *teaches*.

直积结果 instructor $\times$ teaches 的关系模式为:

$$\begin{aligned} \text{instructor} \times \text{teaches} = (&\text{instructor.ID, instructor.name, instructor.dept\_name, instructor.salary} \\ &\text{teaches.ID, teaches.course\_id, teaches.sec\_id, teaches.semester, teaches.year}) \\ = (&\text{instructor.ID, name, dept\_name, salary, teaches.ID, course\_id, sec\_id, semester, year}) \end{aligned}$$

对于那些仅出现在其中一个关系模式中的属性，我们通常省略关系名前缀，这种简化不会导致歧义.
从上述命名规范可知，参与直积运算的两个关系必须具有不同的名称.
这一要求在某些情况下会导致问题，例如当需要一个关系与其自身做直积时 (此时需要使用更名运算解决冲突)

1.3.5 更名

更名运算用记号 ρ 表示:

$$\rho_{X(A_1,\ldots,A_n)}(E)$$

其中 E 为关系代数表达式，X 为关系名，$A_1,\ldots,A_n$ 为属性名.

1.3.6 连接

连接运算使我们能够将选择运算和直积运算合并到单个运算中.
设 R,S 为两个关系模式，r 和 s 为对应的关系实例，θ 为 $R\cup S$ 模式的属性上的一个谓词.
则我们可将连接运算定义为:

$$r\bowtie_\theta s:=\sigma_\theta(r\times s)$$

因此 $\sigma_{\text{instructor.ID=teacher.ID}}(\text{instructor}\times\text{teaches})$ 可等价表示为 $\text{instructor}\bowtie_{\text{instructor.ID=teacher.ID}}\text{teaches}$

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

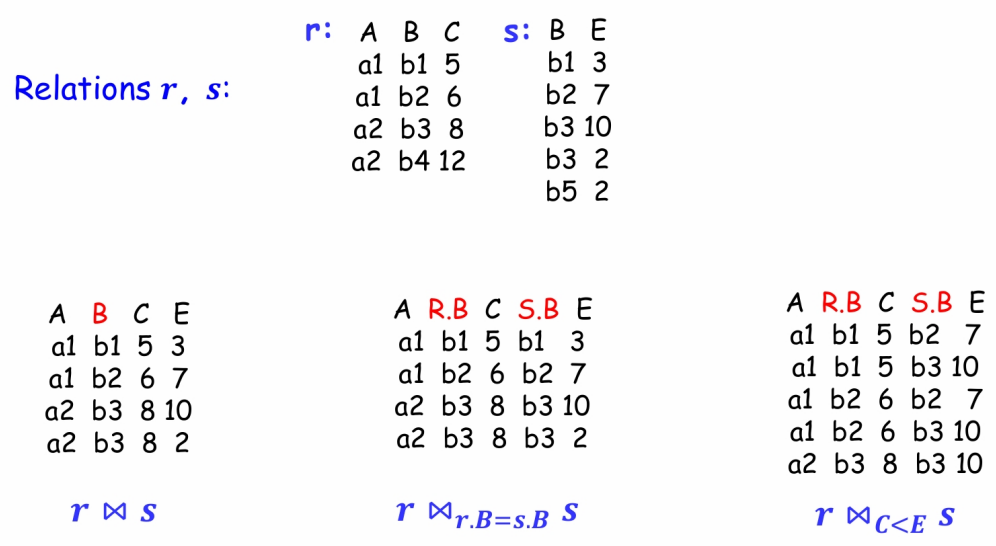
ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure 2.7 The *teaches* relation.

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

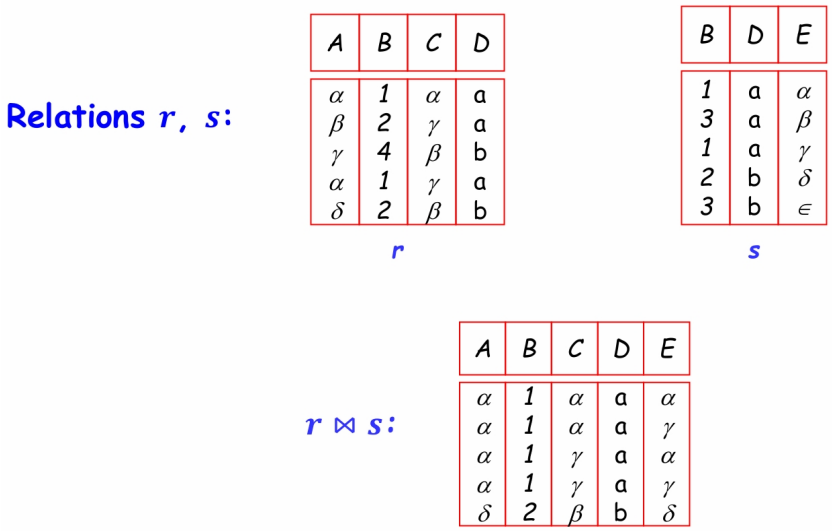
Figure 2.13 Result of $\sigma_{instructor.ID=teaches.ID}(instructor\times teaches)$.

周水庚老师 PPT 上的例子:



自然连接是连接操作的特例，它基于两个关系中所有同名属性之间的等值条件自动执行连接.
它可以保证结果中没有同名属性.
不过缺点也很明显: 如果两张表没有同名属性，则自然连接等同于直积，这可能会带来风险.
我们可以将**自然连接** (natural join) 记为:

$$r \bowtie s := \prod_{\text{去除重复的公共属性}} \sigma_{\text{公共属性上值相等}} (r \times s)$$



1.3.7 外连接

在对关系做自然连接时，如果我们保留那些原本要被丢弃的元组 (这可以避免信息的丢失)，
并在这些元组新增加的属性上填入空值 `NULL`，则称之为**外连接** (outer join)
考虑关系表 W_1, W_2 :

W_1 :

name	house	wand_length
H. J. Potter	Gryffindor	11.0
H. J. Granger	Gryffindor	10.75
R. B. Weasley	Gryffindor	14.0

W_2 :

name	house	wand_arm
H. J. Potter	Gryffindor	right
D. L. Malfoy	Slytherin	right
T. M. Riddle	Slytherin	right

- 自然连接 $W_1 \bowtie W_2$ 的查询结果:

$W_1 \bowtie W_2 :=$

name	house	wand_length	wand_arm
H. J. Potter	Gryffindor	11.0	right

- **左外连接** (left outer join) $W_1 ⟕ W_2$:
  返回左表的所有元组以及右表中匹配的部分.
  如果右表没有匹配，则结果中右表的部分将为空值 `NULL`

$W_1 ⟕ W_2 :=$

name	house	wand_length	wand_arm
H. J. Potter	Gryffindor	11.0	right
H. J. Granger	Gryffindor	10.75	NULL
R. B. Weasley	Gryffindor	14.0	NULL

- **右外连接** (right outer join) $W_1 ⟖ W_2$:
  返回右表的所有元组以及左表中匹配的部分.
  如果左表没有匹配，则结果中左表的部分将为空值 `NULL`

$W_1 ⟖ W_2 :=$

name	house	wand_length	wand_arm
H. J. Potter	Gryffindor	11.0	right
D. L. Malfoy	Slytherin	NULL	right
T. M. Riddle	Slytherin	NULL	right

- **全外连接** (full outer join) $W_1 ⟗ W_2$:
  返回两个表中的所有元组以及匹配的部分.
  如果左表没有匹配，则结果中左表的部分将为空值 `NULL`
  如果右表没有匹配，则结果中右表的部分将为空值 `NULL`

$W_1 ⟗ W_2 :=$

name	house	wand_length	wand_arm
H. J. Potter	Gryffindor	11.0	right
H. J. Granger	Gryffindor	10.75	NULL
R. B. Weasley	Gryffindor	14.0	NULL
D. L. Malfoy	Slytherin	NULL	right
T. M. Riddle	Slytherin	NULL	right

周水庚老师 PPT 上的例子:

loan-number	branch-name	amount
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *loan*

customer-name	loan-number
Jones	L-170
Smith	L-230
Hayes	L-155

Relation *borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

Inner Join: *loan* ⋈ *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*

Left Outer Join: *loan* ⟕ *Borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	*null*	*null*	Hayes

Right Outer Join: *loan* ⟖ *borrower*

loan-number	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	*null*
L-155	*null*	*null*	Hayes

Full Outer Join: *loan* ⟗ *borrower*

1.3.8 半连接

在对关系做**自然连接**时，
如果我们将自然连接的结果投影到左表或右表的属性集上，则称之为**半连接** (semi-join)

- **左半连接** (left semi-join) $W_1 ⋉ W_2$:
  只返回自然连接结果在左表属性集上的投影.

$W_1 \ltimes W_2 :=$

name	house	wand_length
H. J. Potter	Gryffindor	11.0

- **右半连接** (right semi-join) $W_1 \rtimes W_2$:
  只返回自然连接结果在右表属性集上的投影.

$W_1 \rtimes W_2 :=$

name	house	wand_arm
H. J. Potter	Gryffindor	right

1.3.9 除

给定关系 $r(R)$ 和 $s(S)$，假设 $S \subseteq R$ (即模式 S 的所有属性都在模式 R 中)
我们可以将**除** (division) 操作定义为:

$$r \div s := \{x : (x, y) \in r \text{ for all } y \in s\}$$

其中 x 是关系 r 的元组 (x, y) 在模式 R 的独有属性集 (记为 $A_{\text{unique}} := R - S$) 上的取值，
而 y 是关系 r 的元组 (x, y) 在模式 R 与模式 S 的公共属性集 (记为 $A_{\text{shared}} := R \cap S$) 上的取值.

除操作对于表示特定形式的查询非常有用，
例如 **"找出报了所有课程的学生"** 或者 **"找出能供应所有种类零件的供应商"**.
但由于它的使用频率不高，因此数据库系统没有设置独立的操作符来实现除操作，
而是通过其他操作的组合来实现除操作:

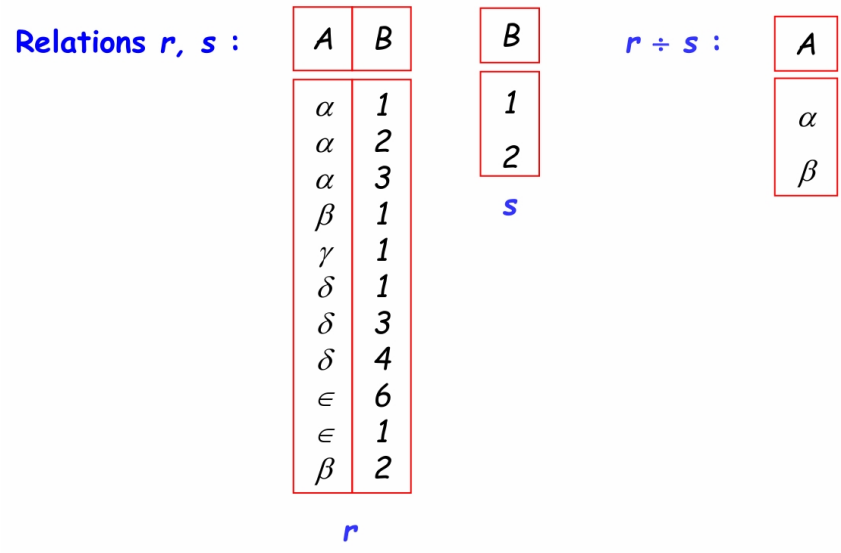
- 首先我们找出 $r \div s$ 的补集，这个补集中的 x 与关系 s 中某个 y 的组合 (x, y) 不在关系 r 中.
  这个补集可以表示为:

$$\Pi_{A_{\text{unique}}}\left(\left(\Pi_{\text{unique}}(r) \times s\right) - r\right)$$

- 然后我们就得到了 $r \div s$，这个集合中的 x 与关系 s 中任意 y 的组合 (x, y) 都在关系 r 中.
  于是 $r \div s$ 可以表示为:

$$r \div s := \Pi_{A_{\text{unique}}}(r) - \Pi_{A_{\text{unique}}}\left(\left(\Pi_{\text{unique}}(r) \times s\right) - r\right)$$

周水庚老师 PPT 上的例子:



Find all customers who have an account at all branches located in Shanghai

$$\Pi_{customer\_name,branch\_name}(depositor \bowtie account) \div \Pi_{branch\_name}(\sigma_{branch\_city="Shanghai"}(branch))$$

1.3.10 赋值运算

借助赋值操作可以把复杂的关系表达式分解成几个简单的步骤.
但过程中生成的中间关系会带来存储代价.

Assignment Operation

The assignment operation (←) provides a convenient way to express complex queries

- Write query as a sequential program consisting of
  - a series of assignments
  - followed by an expression whose value is displayed as a result of the query
- Assignment must be made to a temporary relation variable

Example: write $r \div s$ as:

- $temp_1 \leftarrow \Pi_{R-S}(r)$
- $temp_2 \leftarrow \Pi_{R-S}((temp_1 \times s) - \Pi_{R-S,S}(r))$
- $result = temp_1 - temp_2$

The result to the right of the ← is assigned to the relation variable on the left of the ←

1.3.11 聚合函数

Aggregate Functions and Operations

Aggregation function takes a collection of values and returns a single value as a result

- **avg:** average value
- **min:** minimum value
- **max:** maximum value
- **sum:** sum of values
- **count:** number of values

Aggregate operation in relational algebra

- $_{G_1,G_1,\ldots,G_n}g_{F_1(A_1),F_2(A_2),\ldots,F_n(A_n)}(E)$
  - E is any relational-algebra expression
  - $G_1, G_2, \ldots, G_n$ is a list of attributes on which to group (can be empty)
  - Each F_i is an aggregate function
  - Each A_i is an attribute name

Result of aggregation does not have a name

- Can use rename operation to give it a name
- For convenience, we permit renaming as part of aggregate operation

branch_name $\mathcal{G}$ *sum(balance) as sum_balance* (*account*)

周水庚老师 PPT 上的例子:

Relation r :

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$g_{sum(c)}(r)$:

Sum(C)
27

Relation account grouped by *branch_name*:

branch_name	account_number	balance
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

branch_name $g_{sum(balance)}$ (account)

branch_name	balance
Perryridge	1300
Brighton	1500
Redwood	700

1.3.12 应用举例

同一查询的不同关系表达式的效率是不同的:

Find the names of all customers who have a loan at the Perryridge branch

Query 1

$$\Pi_{customer\_name}(\sigma_{branch\_name = \text{“Perryridge”}}(\sigma_{borrower.loan\_number = loan.loan\_number}(borrower \times loan)))$$

Query 2

$$\Pi_{customer\_name}(\sigma_{loan.loan\_number = borrower.loan\_number}((\sigma_{branch\_name = \text{“Perryridge”}}(loan)) \times borrower))$$

Find all the customers who have accounts from at least the “Downtown” and the “Uptown” branches

Query 1

- $\Pi_{CN}(\sigma_{BN="Downtown"}(depositor \bowtie account)) \cap \Pi_{CN}(\sigma_{BN="Uptown"}(depositor \bowtie account))$
- where CN denotes customer_name and BN denotes branch_name

Query 2

- $\Pi_{customer\_name,branch\_name}(depositor \bowtie account) \div \rho_{temp(branch\_name)}(\{(\text{“}Downtown\text{”}), (\text{“}Uptown\text{”})\})$
- Note that Query2 uses a constant relation

对于某些查询，考虑问题的反面可以方便许多:

Find the largest account balance

Strategy:

- Find those balances that are not the largest
- **Rename** account relation as d so that we can compare each account balance with all others
- Use set difference to find those account balances that were not found in the earlier step

$$\Pi_{balance}(account) - \Pi_{account.balance}(\sigma_{account.balance < d.balance}(account \times \rho_{d}(account)))$$

1.4 数据库操作

1.4.1 空值

Null Values

It is possible for tuples to have a null value for some of their attributes

- null signifies an unknown value or that a value does not exist

The result of any arithmetic expression involving null is null

Aggregate functions simply ignore null values

- Is an arbitrary decision? Could have returned null as result instead?
- We follow the semantics of SQL in its handling of null values

For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same

- Alternative: assume each null is different from each other
- Both are arbitrary decisions, so we simply follow SQL

三值逻辑的运算规则如下:

Three-valued logic (三值逻辑) using the truth value unknown

- **OR:** (unknown or true) = true,
  (unknown or false) = unknown
  (unknown or unknown) = unknown
- **AND:** (true and unknown) = unknown,
  (false and unknown) = false,
  (unknown and unknown) = unknown
- **NOT:** (not unknown) = unknown
- **In SQL** "P is unknown" evaluates to true if predicate P evaluates to unknown

Result of select predicate is treated as false if it evaluates to unknown

1.4.2 删除

Deletion

A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the **selected tuples** are removed from the database

- Can only delete whole tuples
- cannot delete values on particular attributes

A **deletion** is expressed in relational algebra by:

- $r \leftarrow r - E$, where r is a relation and E is a relational algebra query

周水庚老师 PPT 上的例子:

Delete all account records in the Perryridge branch.

$$account \leftarrow account - \sigma_{branch\_name = "Perryridge"}(account)$$

Delete all loan records with amount in the range of 0 to 50

$$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$$

Delete all accounts at branches located in Shanghai

$$r_1 \leftarrow \sigma_{branch\_city = "Shanghai"}(account \bowtie branch)$$
$$r_2 \leftarrow \Pi_{account\_number,\ branch\_name,\ balance}(r_1)$$
$$r_3 \leftarrow \Pi_{customer\_name,\ account\_number}(r_2 \bowtie depositor)$$
$$account \leftarrow account - r_2$$
$$depositor \leftarrow depositor - r_3$$

1.4.3 插入

Insertion

To **insert** data into a relation, we either:

- specify a tuple to be inserted
- write a query whose result is a set of tuples to be inserted

In relational algebra, an **insertion** is expressed by:

- $r \leftarrow r \cup E$, where r is a relation and E is a relational algebra expression.
- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple

周水庚老师 PPT 上的例子:

Insert information in the database specifying that Smith has $1200 in account A_973 at the Perryridge branch.

$$account \leftarrow account \cup \{(A\_973, \text{"Perryridge"}, 1200)\}$$
$$depositor \leftarrow depositor \cup \{(\text{"Smith"}, A\_973)\}$$

Provide as a gift for all loan customers in the Perryridge branch, a $200 savings account. Let the loan number serve as the account number for the new savings account.

$$r_1 \leftarrow (\sigma_{branch\_name = \text{"Perryridge"}} (borrower \bowtie loan))$$
$$account \leftarrow account \cup \Pi_{loan\_number,\ branch\_name,\ 200} (r_1)$$
$$depositor \leftarrow depositor \cup \Pi_{customer\_name,\ loan\_number}(r_1)$$

1.4.4 更新

Updating

A mechanism to change a value in a tuple without changing all other values in the tuple

Use the **generalized projection** operator to do this task

- $r \leftarrow \Pi_{F_1, F_2, \ldots, F_i}(r)$
- Each F_i is either
  - the ith attribute of r, if the ith attribute is not updated, or
  - if the attribute to be updated, F_i is an expression, involving only constants and the attributes of r, which gives the new value for the attribute

To select some tuples from r to update, we can use the following expression:

$$r \leftarrow \Pi_{F_1, F_2, \ldots, F_n}(\sigma_P(r)) \cup (r - \sigma_P(r))$$

where P denotes the selection condition that chooses which tuples to update

周水庚老师 PPT 上的例子:

Make interest payments by increasing all balances by 5 percent.

$$account \leftarrow \Pi_{\ AN,\ BN,\ BAL * 1.05} (account)$$

where AN, BN and BAL stand for *account_number*, *branch_name* and *balance*, respectively

Pay all accounts with balances over $10,000 6 percent interest and pay all others 5 percent

$$account \leftarrow \Pi_{\ AN,\ BN,\ BAL * 1.06} (\sigma_{\ BAL > 10000} (account)) \cup \Pi_{AN,\ BN,\ BAL * 1.05} (\sigma_{BAL \le 10000} (account))$$

1.4.5 视图

视图 (view) 不作为关系存在数据库中，只有在查看时才会根据关系表达式生成.

Views（视图）

- In some cases, it is not desirable for all users to see the entire logical model
- Consider a person who needs to know a customer's loan number but has no need to see the loan amount. This person should see a relation described, in the relational algebra, by

$$\Pi_{customer\_name,loan\_number,branch\_name}(borrower \bowtie loan)$$

- Any relation that is not of the conceptual model but is made visible to a user as a "**virtual relation**" is called a **view**
- A view is defined using the create view statement which has the form

create view v as < query expression >

- Once a view is defined, the **view name** can be used to refer to the **virtual relation** that the view generates
- View definition is not the same as creating a new relation by evaluating the query expression
  - Rather, **a view definition** causes **the saving of an expression**
  - the expression is substituted into queries using the view

周水庚老师 PPT 上的例子:

Consider the view (named *all_customer*) consisting of branches and their customers

create view *all_customer* as

$$\Pi_{branch\_name,\ customer\_name}(depositor \bowtie account)$$
$$\cup\ \Pi_{branch\_name,\ customer\_name}(borrower \bowtie loan)$$

We can find all customers of the Perryridge branch by writing:

$$\Pi_{customer\_name}(\sigma_{branch\_name\ =\ "Perryridge"}(all\_customer))$$

利用已有的视图定义新的视图:

Views Defined Using Other Views

- One view may be used to define another view
- A view relation v_1 is said to depend directly (直接依赖) on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to depend (依赖) on view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be recursive (递归) if it depends on itself
- A way to define the meaning of views defined in terms of other views
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations
- View expansion of an expression repeats the following replacement step:

  repeat
  Find any view relation v_i in e_1
  Replace the view relation v_i by the expression defining v_i
  until no more view relations are present in e_1
- As long as the view definitions are not recursive, this loop will terminate

1.4.6 基于视图的更新

Modification of the Database

- The content of the database may be modified using the following operations:
  - Deletion
  - Insertion
  - Updating
- All these operations are expressed using the assignment operation

关键是使用赋值操作修改数据库中的关系:

Updates Through View

- **Must be translated to modifications of the actual relations**
- Consider the person who needs to see all loan data in the loan relation except amount. The view given to the person, *branch_loan*, is defined as:

$$\textit{create view branch\_loan as } \Pi_{branch\_name, loan\_number}(loan)$$

- Since we allow a view name to appear wherever a relation name is allowed, the person may write:

$$branch\_loan \leftarrow branch\_loan \cup \{(\text{"Perryridge"}, L\_37)\}$$

- An insertion into relation *loan* requires a value for amount. The insertion can be handled by either.
  - rejecting the insertion
  - inserting a tuple (L_37, "Perryridge", null)
- Some updates through views are impossible to translate into database relation updates

$$create\ view\ v\ as\ (\sigma_{branch\_name="Perryridge"}(account)$$
$$v \leftarrow v \cup (L\_99, \text{"Downtown"}, 23)$$

- Others cannot be translated uniquely

create view *all_customer* as

$$\Pi_{branch\_name,\ customer\_name}\ (depositor \bowtie account)$$
$$\cup\ \Pi_{branch\_name,\ customer\_name}\ (borrower \bowtie loan)$$

$$all\_customer \leftarrow all\_customer \cup \{(\text{Perryridge}, \text{"}John\text{"})\}$$

  - Have to choose loan or account, and create a new loan/account number

The End